

Kubernetes and Cloud Native Associate - KCNA

Scheduling

Multiple Schedulers

Welcome to this article on deploying multiple schedulers in a Kubernetes cluster. In this guide, we will walk through configuring and deploying additional schedulers—enabling you to use custom placement logic for specific applications while still relying on the default scheduler for most workloads.

Kubernetes distributes pods evenly across nodes and considers factors such as taints, tolerations, and node affinity when using its default scheduler. However, if your application requires custom scheduling behavior, Kubernetes allows you to implement and deploy a custom scheduler alongside the default one. When running multiple schedulers, each must have a unique name so Kubernetes can easily differentiate between them. The default scheduler is typically named "default-scheduler".



Note

Even though you do not need an explicit configuration file for the default scheduler, creating one can help you document and customize the scheduling behavior if needed.

Default Scheduler Configuration

A configuration file for the default scheduler might look like this:

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: default-scheduler
```

Creating a Custom Scheduler

To create a custom scheduler, prepare a separate configuration file that specifies a unique scheduler name. For example:

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler
```

Deploying an Additional Scheduler as a Service

When deploying an extra scheduler as a service, you typically use the same kube-scheduler binary or a modified version with a unique configuration file. Below are two examples—one for the default scheduler and another for a custom scheduler.

Default Scheduler Service

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.12.0/bin/linux/

ExecStart=/usr/local/bin/kube-scheduler \
--config=/etc/kubernetes/config/kube-scheduler.yaml
```

Custom Scheduler Service

Create a service file for your custom scheduler, for example, `my-scheduler-2.service`:

```
# my-scheduler-2.service
ExecStart=/usr/local/bin/kube-scheduler \
--config=/etc/kubernetes/config/my-scheduler-2-config.yaml
```

And the corresponding configuration file:

```
# my-scheduler-2-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler-2
```

Each scheduler reads its configuration file—which includes its unique `schedulerName`—along with any additional options like the kubeconfig file necessary for connecting to the Kubernetes API.

Deploying a Scheduler as a Pod

Another common approach is deploying the scheduler as a pod. In this setup, define a pod manifest that points to the custom kube-scheduler configuration file. Below is an example of a basic pod definition:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-custom-scheduler
  namespace: kube-system
spec:
  containers:
  - name: kube-scheduler
    image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
    command:
```

- kube-scheduler
- --address=127.0.0.1
- --kubeconfig=/etc/kubernetes/scheduler.conf

The custom scheduler configuration file might look as follows:

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler
```

Leader Election for High Availability

When running multiple instances of the same scheduler (such as on different master nodes), enable leader election to ensure only one instance is active at a time. The following pod manifest and configuration file enable leader election:

Pod manifest with leader election enabled:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-custom-scheduler
  namespace: kube-system
spec:
  containers:
  - name: kube-scheduler
    image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
    command:
      - kube-scheduler
      - --address=127.0.0.1
      - --kubeconfig=/etc/kubernetes/scheduler.conf
      - --config=/etc/kubernetes/my-scheduler-config.yaml
```

And the corresponding configuration file:

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: my-scheduler
  leaderElection:
    leaderElect: true
    resourceNamespace: kube-system
    resourceName: lock-o
```

This configuration differentiates your custom scheduler's leader election mechanism from that of the default scheduler, ensuring seamless high availability in a multi-master setup.

Deploying a Scheduler as a Deployment

In many Kubernetes environments, control plane components are deployed as pods or Deployments using kubeadm. You can also deploy your custom scheduler in this manner by building a custom Docker image.

Building a Custom Scheduler Image

Clone the Kubernetes repository and build your scheduler:

```
git clone https://github.com/kubernetes/kubernetes.git
cd kubernetes
make
```

Then create a Dockerfile that includes the scheduler binary:

```
FROM busybox
ADD ./_output/local/bin/linux/amd64/kube-scheduler /usr/local/bin/kube-scheduler
```

Build and push your Docker image:

```
docker build -t gcr.io/my-gcp-project/my-kube-scheduler:1.0 .  
gcloud docker -- push gcr.io/my-gcp-project/my-kube-scheduler:1.0
```

RBAC Setup for the Custom Scheduler

For secure deployment, create a ServiceAccount and bind the necessary ClusterRoles. Here's an example configuration:

```
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  name: my-scheduler  
  namespace: kube-system  
---  
apiVersion: rbac.authorization.k8s.io/v1  
kind: ClusterRoleBinding  
metadata:  
  name: my-scheduler-as-kube-scheduler  
subjects:  
  - kind: ServiceAccount  
    name: my-scheduler  
    namespace: kube-system  
roleRef:  
  kind: ClusterRole  
  name: system:kube-scheduler  
  apiGroup: rbac.authorization.k8s.io  
---  
apiVersion: rbac.authorization.k8s.io/v1  
kind: ClusterRoleBinding  
metadata:  
  name: my-scheduler-as-volume-scheduler  
subjects:  
  - kind: ServiceAccount  
    name: my-scheduler  
    namespace: kube-system  
roleRef:  
  kind: ClusterRole
```

```
name: system:volume-scheduler
apiGroup: rbac.authorization.k8s.io
```

Deployment Manifest Example

Below is a sample Deployment manifest for your custom scheduler. This deployment uses a ConfigMap to mount the custom configuration file as a volume:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-scheduler
  namespace: kube-system
  labels:
    component: scheduler
    tier: control-plane
spec:
  replicas: 1
  selector:
    matchLabels:
      component: scheduler
      tier: control-plane
  template:
    metadata:
      labels:
        component: scheduler
        tier: control-plane
        version: second
    spec:
      serviceAccountName: my-scheduler
      containers:
        - name: kube-second-scheduler
          image: gcr.io/my-gcp-project/my-kube-scheduler:1.0
          command:
            - /usr/local/bin/kube-scheduler
            - --config=/etc/kubernetes/my-scheduler/my-scheduler-config.yaml
          livenessProbe:
            httpGet:
              path: /healthz
```

```
    port: 10259
    scheme: HTTPS
    initialDelaySeconds: 15
  readinessProbe:
    httpGet:
      path: /healthz
      port: 10259
      scheme: HTTPS
  resources:
    requests:
      cpu: "0.1"
  securityContext:
    privileged: false
  volumeMounts:
    - name: config-volume
      mountPath: /etc/kubernetes/my-scheduler
  hostNetwork: false
  hostPID: false
  volumes:
    - name: config-volume
      configMap:
        name: my-scheduler-config
```

A corresponding ConfigMap that contains the scheduler configuration is as follows:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-scheduler-config
  namespace: kube-system
data:
  my-scheduler-config.yaml: |
    apiVersion: kubescheduler.config.k8s.io/v1beta2
    kind: KubeSchedulerConfiguration
    profiles:
      - schedulerName: my-scheduler
    leaderElection:
      leaderElect: false
```


For enhanced security, ensure your RBAC resources are correctly configured. Below is an example of a ClusterRole setup:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: system:kube-scheduler
  annotations:
    rbac.authorization.kubernetes.io/autoupdate: "true"
  labels:
    kubernetes.io/bootstrapping: rbac-defaults
rules:
- apiGroups:
  - coordination.k8s.io
  resources:
  - leases
  verbs:
  - create
- apiGroups:
  - coordination.k8s.io
  resources:
  - kube-scheduler
  verbs:
  - get
  - list
  - watch
```

When you run:

```
kubectl get pods --namespace=kube-system
```

You should see your custom scheduler pod or Deployment along with the other control plane components.

Using the Custom Scheduler for Pods

Once your custom scheduler is deployed, you can instruct specific pods to use it by setting the `schedulerName` field in their manifests. For example, here is a pod manifest for an nginx pod using the custom scheduler:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx
  schedulerName: my-custom-scheduler
```

Deploy the pod with:

```
kubectl create -f nginx-pod.yaml
```

If the custom scheduler is not configured correctly, the pod will remain in a pending state. Use the following commands to inspect pod events and troubleshoot:

```
kubectl describe pod nginx
kubectl get events -o wide
```

A typical events output might be:

LAST SEEN	COUNT	NAME	KIND	TYPE	REASON	SOURCE	MESSAGE
9s	1	nginx.15	Pod	Normal	Scheduled	my-custom-scheduler	Successfully assigned default/nginx to node01
8s	1	nginx.15	Pod	Normal	Pulling	kubelet, node01	pulling image "nginx"

LAST SEEN	COUNT	NAME	KIND	TYPE	REASON	SOURCE	MESSAGE
2s	1	nginx.15	Pod	Normal	Pulled	kubelet, node01	Successfully pulled image "nginx"
2s	1	nginx.15	Pod	Normal	Created	kubelet, node01	Created container
2s	1	nginx.15	Pod	Normal	Started	kubelet, node01	Started container

The event source confirms that the custom scheduler successfully handled pod scheduling.



Troubleshooting

To troubleshoot your custom scheduler, view its logs with:

```
kubectl logs my-custom-scheduler --namespace=kube-system
```

Reviewing these logs will help pinpoint configuration errors or leader election issues.

Conclusion

Deploying multiple schedulers in a Kubernetes cluster allows you to implement tailored scheduling strategies for different workloads while leaving the default scheduler in place. This guide outlined the configurations and deployment methods—ranging from running the scheduler as a service or pod to deploying it as a Deployment with proper RBAC and leader election setups.

Happy scheduling!

For more information on Kubernetes scheduling, visit the [Kubernetes Documentation](#) and explore additional resources on [Custom Schedulers](#).



Watch Video

Watch video content

Previous

◀ Static Pods

Next

Configuring Kubernetes Scheduler Profiles ▶